

```

import java.util.ArrayList;

public class EqualSum {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 3, 4 };
        int target = 4;
        find(arr, 0, new ArrayList<Integer>(), new ArrayList<ArrayList<Integer>>(), target);
    }

    static void find(int[] arr, int index, ArrayList<Integer> list, ArrayList<ArrayList<Integer>> all,
int target) {

        if (target == 0) {
            boolean consit = true;

            if(consit) {
                all.add(list);
            }
            System.out.println(list);
            return;
        }

        if (index == arr.length) {
            return;
        }

        list.add(arr[index]);
        find(arr, index + 1, list, all, target - arr[index]);

        list.remove(list.size() - 1);
        find(arr, index + 1, list, all, target);
    }
}

```

```

import java.util.Arrays;

public class FindLeftmaxOfEachElement {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 0, 3, 8, 3, 4, 10 };
        find(arr);
    }

    static void find(int[] arr) {
        int len = arr.length;
        int[] left = new int[len];
        left[0] = arr[0];

        for (int i = 1; i < len; i++) {
            left[i] = Math.max(arr[i], left[i - 1]);
        }
    }
}

```

```

    }

    System.out.println(Arrays.toString(left));

}

3.import java.util.HashMap;
import java.util.HashSet;
import java.util.Set;

public class FindMaxTime {

    static int maxHour = Integer.MIN_VALUE;
    static int maxMinutes = Integer.MIN_VALUE;
    static HashMap<Integer, Integer> map = new HashMap<>();

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        String s = "02:05"; // 21: 51

        find("", s.substring(0, 2) + s.substring(3));

        System.out.println(map);
    }

    static void find(String s, String rem) {

        if (rem.length() == 0) {
            int hour = Integer.parseInt(s.substring(0, 2));
            int min = Integer.parseInt(s.substring(2));

            if (hour < 24 && min < 60) {
                if (hour >= maxHour) {
                    maxHour = hour;

                    if (map.containsKey(hour)) {
                        int minval = map.get(hour);
                        if (min > minval)
                            map.put(hour, min);
                    } else {
                        map.clear();
                        map.put(hour, min);
                    }
                }
            }

            }

        // System.err.println(s);
        return;
    }

    for (int i = 0; i < rem.length(); i++) {
        String val = s + rem.charAt(i);
        int valofint = val.charAt(0);

        find(s + rem.charAt(i), rem.substring(0, i) + rem.substring(i + 1));
    }
}

```

```

    }
}

4.import java.util.HashSet;
import java.util.Set;

public class FindUniqueValues {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 2, 3, 3, 2, 1 };
        find(arr);

    }

    static void find(int[] arr) {
        Set<Integer> s = new HashSet<>();

        for (int i = 0; i < arr.length; i++) {
            s.add(arr[i]);
        }

        for (int x : s) {
            System.out.println(x);
        }

    }

}

```

```

5.
public class Jump11 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 3, 9, 8, 5, 2, 6, 7, 6, 8, 9 };

        find(arr);

    }

    static void find(int[] arr) {

        int a = arr[0];
        int b = arr[0];
        int jump = 1;

        for (int i = 1; i < arr.length; i++) {

            a--;
            b--;
            if (arr[i] > b) {
                b = arr[i];
            }

            if (a == 0) {
                a = b;
                jump++;
            }
        }
    }
}

```

```

        }

    }

    System.out.println(jump);

}
}

```

6.

```

public class LongestPalindomicSubString {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        String s = "babad";
        System.out.println(SubStringFind(s));

    }

    static String SubStringFind(String s) {
        int max = Integer.MIN_VALUE;
        String sub = "";
        int start = 0;
        int end = s.length() - 1;

        // while (start < end) {
        //     if (s.charAt(start) == s.charAt(end)) {
        //         boolean ispali = isPalindrome(s, start, end);
        //         if (ispali && start - end > max) {
        //             max = end - start;
        //             sub = s.substring(start, end + 1);
        //         }
        //     }
        //     start++;
        //     end--;
        // }

        for (int i = 0; i < s.length(); i++) {
            for (int j = i; j < s.length(); j++) {
                boolean ispali = s.charAt(j) == s.charAt(i) ? isPalindrome(s, i, j) : false;
                if (ispali && j - i > max) {
                    max = j - i;
                    sub = s.substring(i, j + 1);
                }
            }
        }

        return sub;
    }

    static boolean isPalindrome(String s, int start, int end) {
        while (start < end) {
            if (s.charAt(start) != s.charAt(end)) {
                return false;
            }
        }
    }
}

```

```

        start++;
        end--;
    }
    return true;
}
}

```

7

```

import java.util.Stack;

public class LongestValidParanthesis {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String s = "(";
        System.out.println(find(s));
    }

    static int find(String s) {
        Stack<Character> st = new Stack<>();
        int len = s.length();
        int count = 0;
        for (int i = 0; i < len; i++) {
            char c = s.charAt(i);
            if (c == '(') {
                st.push(c);
            } else {
                if (!st.isEmpty()) {
                    st.pop();
                } else {
                    count++;
                }
            }
        }

        System.out.println(st);
        return len - (count + st.size());
    }
}

```

8.

```

import java.util.ArrayList;
import java.util.List;

public class MatrixPath {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr[] = { { 1, 2, 3, 4},
                        { 4, 5, 6, 7},
                        {8,9,10,11} };

        List<Integer> list = new ArrayList<>();
        list.add(1);
        List<Integer> minList = new ArrayList<>();
        find(arr, 0, 0, arr[0].length, list, minList);
    }
}

```

```

        System.out.println(minList + " " + (minList.size()-1));
    }

    static void find(int[][] arr, int row, int col, int rowSize, int colSize, List<Integer> list,
        List<Integer> minList) {

        if (row == rowSize - 1 && col == colSize - 1) {
            if (minList.size() < list.size()) {
                minList.clear();
                minList.addAll(list);
            }
            System.out.println(list);
            return;
        }
        if (row >= rowSize || col >= colSize) {
            return;
        }

        if (row < rowSize - 1) {
            list.add(arr[row + 1][col]);
            find(arr, row + 1, col, rowSize, colSize, list, minList);
            list.remove(list.size() - 1);
        }

        if (col < colSize - 1) {
            list.add(arr[row][col + 1]);
            find(arr, row, col + 1, rowSize, colSize, list, minList);
            list.remove(list.size() - 1);
        }

    }
}

```

9.

```
import java.util.ArrayList;
```

```
public class MinCoins {
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 3, 2, 5 };
        int target = 11;
        ArrayList<Integer> minimumList = new ArrayList<>();
        find(arr, target, new ArrayList<>(), minimumList);
        System.out.println(minimumList + " " + minimumList.size());
    }

```

```

    static void find(int[] arr, int target, ArrayList<Integer> coinList, ArrayList<Integer>
minimumList) {

```

```

        if (target == 0) {

            if (minimumList.size() == 0 || coinList.size() < minimumList.size()) {
                minimumList.clear();
                minimumList.addAll(coinList);
            }
        }
    }
}

```

```

        return;
    }
    System.out.println(coinList);
}
if (target < 0) {
    return;
}

for (int i = 0; i < arr.length; i++) {

    coinList.add(arr[i]);
    find(arr, target - arr[i], coinList, minimumList);
    coinList.remove(coinList.size() - 1);

}

}
}

```

10

```

public class MinNumCharctersToAddMakePalindrome {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String s = "race";
        System.out.println(find(s));
    }

    static String find(String s) {

        String op = "";

        for (int i = s.length() - 1; i >= 0; i--) {
            op = s.substring(i);
            op = rev(op);

            if (isPalindrome(op + s)) {
                return op + s;
            }
        }

        return "";
    }

    static String rev(String s) {
        char[] rev = s.toCharArray();
        int start = 0;
        int end = rev.length - 1;
        while (start < end) {
            char temp = rev[start];
            rev[start] = rev[end];
            rev[end] = temp;
            start++;
            end--;
        }
        return String.valueOf(rev);
    }
}

```

```

static boolean isPalindrome(String s) {

    int start = 0;
    int end = s.length() - 1;
    while (start < end) {
        if (s.charAt(start) != s.charAt(end)) {
            return false;
        }
        start++;
        end--;
    }

    return true;
}

}

11.
public class PrintAllSubString {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String s = "abcdef";
        find(s, 0);

    }

    static void find(String s, int index) {
        if (index == s.length()) {
            return;
        }

        for (int i = index; i < s.length(); i++) {

            System.out.println(s.substring(index, i + 1));

        }

        find(s, index + 1);
    }

}

```

12.

```

public class PrintPermutations {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        String s = "abc";
        print("", s);

    }

    static void print(String s, String rem) {
        if (rem.length() == 0) {
            System.out.println(s);
            return;
        }
    }
}

```



```

        for (int i = 0; i < rem.length(); i++) {
            print(s + rem.charAt(i), rem.substring(0, i) + rem.substring(i + 1));
        }
    }
}

```

13.

```

import java.util.ArrayList;
import java.util.HashSet;
import java.util.Set;

```

```

public class PrintUniquePermuations {

```

```

    static ArrayList<String> set = new ArrayList();

```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub

```

```

        String s = "9010";
        find("", s);
        System.out.println(set);

```

```

    }

```

```

    static void find(String s, String rem) {

```

```

        if (rem.length() == 0) {
            // System.out.println(s);
            set.add(s);
            return;
        }

```

```

        for (int i = 0; i < rem.length(); i++) {
            find(s + rem.charAt(i), rem.substring(0, i) + rem.substring(i + 1));
        }

```

```

    }

```

```

}

```

14.

```

public class ProductOfElementExceptThatElement {

```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub

```

```

        int[] arr = { 1, 2, 3, 4, 5 };
        find(arr);

```

```

    }

```

```

    static void find(int[] arr) {

```

```

        int len = arr.length;

```

```

        int left[] = new int[len];
        int right[] = new int[len];

        left[0] = 1;

        for (int i = 1; i < right.length; i++) {
            left[i] = left[i - 1] * arr[i - 1];
        }

        right[len - 1] = 1;
        for (int i = len - 2; i >= 0; i--) {
            right[i] = right[i + 1] * arr[i + 1];
        }

        for (int i = 0; i < arr.length; i++) {
            System.out.print(left[i] * right[i] + " ");
        }

    }
}

```

15.

```
import java.util.Arrays;
```

```

public class RemoveDuplicatesInSortedArray {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 0, 0, 1, 1, 1, 2, 2, 3, 3, 3, 3, 4, 5, 6 };
        remove(arr);
    }

    static void remove(int[] arr) {
        int len = arr.length;
        int counter = 1;
        for (int i = 1; i < len; i++) {
            if (arr[i] != arr[i - 1]) {
                arr[counter] = arr[i];
                counter++;
            }
        }

        for (int i = counter; i < len; i++) {
            arr[i] = -1;
        }
        System.out.println(Arrays.toString(arr));
    }
}

```

16.

```
import java.util.Arrays;
```

```

public class RightMaxOfEachElement {

    public static void main(String[] args) {

```

```

        // TODO Auto-generated method stub

        int[] arr = { 1, 2, 3, 4, 5 };
        find(arr);
    }

    static void find(int[] arr) {

        int len = arr.length;
        int[] right = new int[len];
        right[len - 1] = arr[len - 1];

        for (int i = len - 2; i >= 0; i--) {
            right[i] = Math.max(arr[i], right[i + 1]);
        }

        System.out.println(Arrays.toString(right));
    }
}

17.
import java.util.HashMap;
import java.util.Iterator;

public class SamePattern {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        String s1 = "abba";
        String s2 = "hello java java hell";
        System.out.println(find(s1, s2));
    }

    static boolean find(String s1, String s2) {
        int len = s1.length();
        String[] words = s2.split(" ");
        if (len != words.length) {
            return false;
        }
        HashMap<Character, String> map = new HashMap<>();

        for (int i = 0; i < words.length; i++) {
            char c = s1.charAt(i);
            String currentWord = words[i];

            if (map.containsKey(c)) {
                String mapWord = map.get(c);
                if (!mapWord.equals(currentWord)) {
                    return false;
                }
            }

            map.put(c, currentWord);
        }
    }
}

```

```

        return true;
    }
}

18.
import java.util.HashMap;

public class SubStringWithOutRepeatedCharacter {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String s = "abcdacbddef";
        find(s);
    }

    static void find(String s) {
        HashMap<Character, Integer> map = new HashMap<>();
        int max = Integer.MIN_VALUE;
        int start = 0;
        int end = 0;
        String sub = "";
        for (int i = 0; i < s.length(); i++) {

            if (map.containsKey(s.charAt(i))) {
                start = map.get(s.charAt(i)) + 1;
                map.put(s.charAt(i), i);
            } else {
                map.put(s.charAt(i), i);
                end = i;
            }

            if (end - start + 1 > max) {
                max = end - start + 1;
            }

        }

        System.out.println(max);
        System.out.println(s.substring(start, start + max));
    }
}

```

```

19.
import java.util.Arrays;

public class TrapWater {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 4, 2, 0, 3, 2, 5 };
        find(arr);
    }

    static int find(int[] arr) {

```

```

        int len = arr.length;
        int left[] = new int[len];
        int right[] = new int[len];

        left[0] = arr[0];
        for (int i = 1; i < len; i++) {
            left[i] = Math.max(arr[i], left[i - 1]);
        }

        right[len - 1] = arr[len - 1];
        for (int i = len - 2; i >= 0; i--) {
            right[i] = Math.max(arr[i], right[i + 1]);
        }

        System.out.println(Arrays.toString(left));
        System.out.println(Arrays.toString(right));

        int total = 0;

        for (int i = 0; i < len; i++) {
            total = total + Math.min(left[i], right[i]) - arr[i];
        }
        System.out.println(total);

        return 0;
    }
}

```

21.

```
package practice;
```

```
import java.util.ArrayList;
```

```
public class LongestIncreasingSubSequenceInArray {
    static int max = Integer.MIN_VALUE;
    static ArrayList<Integer> finalList = new
    ArrayList<>();

```

```

        public static void main(String[] args) {
            // TODO Auto-generated method stub
            int[] arr = { 1, 8, 9, 7, 10 };

            System.out.println(find(arr, 0, new
            ArrayList<>()));
            System.out.println(finalList);

        }
    }

```

```

    static int find(int[] arr, int index,
ArrayList<Integer> list) {

        if (index == arr.length) {
            if (list.size() > max) {
                max = list.size();
                finalList = new ArrayList<>(list);
            }
            System.out.println(list);
            return 1;
        }
        int count = 0;

        if (list.size() > 0 &&
list.get(list.size() - 1) < arr[index]) {
            list.add(arr[index]);
            count = find(arr, index + 1, list);
            list.remove(list.size() - 1);
        } else {
            list.add(arr[index]);
            count = find(arr, index + 1, list);
            list.remove(list.size() - 1);
        }

        count = count + find(arr, index + 1,
list);

        return count;
    }
}

```

22.

```
package practice.session24;
```

```
public class ClimbingSteps {
    public static void main(String[] args) {

```

```

        int tar = 3;
        System.out.println(find(0, tar));
    }

    static int find(int step, int target) {
        if (step == target) {
            return 1;
        }
        if (step > target) {
            return 0;
        }

        int left = find(step + 1, target);
        int right = find(step + 2, target);

        return left + right;
    }
}

```

23.

```

package practice.session24;

import java.util.ArrayList;
import java.util.Arrays;

import javax.print.attribute.standard.Media;

public class CoinSum {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 2, 5 };
        int target = 11;
        int[] memory = new int[target + 1];
    }
}

```

```

        System.out.println(find(target, arr, new
ArrayList<>(), memory));

    }

    static int find(int target, int[] arr,
ArrayList<Integer> list, int[] memory) {
        if (target == 0) {
            return 0;
        }
        if (target < 0) {
            return Integer.MAX_VALUE;
        }

        if (memory[target] != 0) {
            return memory[target];
        }
        int minCoins = Integer.MAX_VALUE;
        for (int i = 0; i < arr.length; i++) {
            int count = find(target - arr[i], arr,
list, memory);
            if (count != Integer.MAX_VALUE)
                minCoins = Math.min(count + 1,
minCoins);
        }
        return memory[target] = minCoins;
    }
}

```

24.

```
package practice.session24;
```

```
public class DecodeWays {
```

```
    public static void main(String[] args) {
        // TODO Auto-generated method stub
    }
}

```



```

public static void main(String[] args) {
    // TODO Auto-generated method stub
    int num = 10;
    int[] memory = new int[num + 1];
    System.out.println(find(num, memory));
}

static int find(int num, int[] memory) {
    if (num <= 1) {
        return num;
    }
    if (memory[num] != 0) {
        return memory[num];
    }

    memory[num] = find(num - 1, memory) +
find(num - 2, memory);
    return memory[num];
}
}

```

26.

```
package practice.session24;
```

```
public class HouseRobber {
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 3, 1, 2, 5 };
        int[] memory = new int[arr.length];
        System.out.println(find(arr, 0, arr.length
- 1, memory));
    }
}

```

```

        static int find(int[] arr, int start, int end,
int[] memory) {
            if (start == end) {
                return arr[start];
            }
            if (start + 1 == end) {
                return Math.max(arr[start], arr[end]);
            }
            if (memory[start] != 0) {
                return memory[start];
            }

            int left = arr[start] + find(arr, start +
2, end, memory);
            int right = find(arr, start + 1, end,
memory);
            memory[start] = Math.max(left, right);
            return memory[start];

        }
    }
}

```

27.

```
package practice.session24;
```

```
import java.util.ArrayList;
import java.util.List;
```

```
public class SubSet {
    public static void main(String[] args) {

        int[] arr = { 1, 2, 3 };
        int target = 3;
        find(arr, 0, new ArrayList<>(), target);

    }
}

```

```

    static void find(int[] arr, int index,
List<Integer> list, int target) {

        if (target == 0) {
            System.out.println(list);
            return;
        }
        if (index == arr.length) {
            // System.out.println(list);
            return;
        }

        list.add(arr[index]);
        find(arr, index + 1, list, target -
arr[index]);

        list.remove(list.size() - 1);
        find(arr, index + 1, list, target);

    }

}

```

28.

```

package Session_22;

public class AmToPm12To24 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        String time = "01:30AM";
        find(time);

    }

    static void find(String s) {

```

```

        int extramin = s.indexOf("AM") == -1 ? 720
: 0;

        int hours =
Integer.parseInt(s.substring(0, 2));
        int min = Integer.parseInt(s.substring(3,
5));

        int totalmin = hours * 60 + min +
extramin;

        String hourInString = totalmin / 60 + "";
        String minInString = totalmin % 60 + "";

        System.out.println(hourInString + ":" +
minInString);

    }

}

```

29.

```

package Session_22;

import java.util.ArrayList;

public class SubSetWithTarget {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 1, 2, 2 };
        int tar = 2;
        find(arr, 0, new ArrayList<>(), tar);

    }
}

```

```

    static void find(int[] arr, int index,
ArrayList<Integer> list, int target) {

        if (target == 0) {
            System.out.println(list);
            return;
        }

        if (index == arr.length) {
            return;
        }

        list.add(arr[index]);
        find(arr, index + 1, list, target -
arr[index]);

        list.remove(list.size() - 1);
        find(arr, index + 1, list, target);

    }

}

```

30.

```

package Session_24;

import java.util.Stack;

public class MinimumRemovePeranthisCount {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String s = "(()";

        System.out.println(s.length() - find(s));
    }

    static int find(String s) {

```

```

    int count = 0;
    Stack<Character> stack = new Stack<>();

    for (int i = 0; i < s.length(); i++) {

        char c = s.charAt(i);

        if (c == '(') {
            stack.push('(');
        } else {

            if (stack.isEmpty()) {
                count++;

            } else {
                stack.pop();
            }

        }

    }

    return count + stack.size();
}

```

}

31.

```
package Session_24;
```

```
import java.util.Stack;
```

```
public class ReverseTheWordsString {
```

```
    public static void main(String[] args) {
        // TODO Auto-generated method stub
    }
}

```

```
32.  
package Session_25;  
  
public class DecodeWayMem {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        String s =  
"1111111111111111111111111111111111111111111111111";  
        int memory[] = new int[s.length()];  
        System.out.println(find(s, 0, memory));  
    }  
}
```



```

    static int find(String s, int index, int[]
memory) {

        if (index == s.length()) {
            return 1;
        }

        if (s.charAt(0) == '0') {
            return 0;
        }

        if (memory[index] != 0) {
            return memory[index];
        }

        int left = find(s, index + 1, memory);

        int right = 0;

        if (index < s.length() - 1 &&
Integer.parseInt(s.substring(index, index + 2)) <=
26) {
            right = find(s, index + 2, memory);
        }

        memory[index] = left + right;

        return memory[index];
    }
}

```

33.

package Session_26;

import java.util.ArrayList;

```

public class CanFindSum {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 3, 6 };
        call(arr);
    }

    static void call(int[] arr) {
        int sum = 0;
        for (int i = 0; i < arr.length; i++) {
            sum = sum + arr[i];
        }

        if (sum % 2 == 1) {
            System.out.println("we cant find");
        } else {
            boolean res = find(arr, 0, sum / 2,
new ArrayList<>());
            if (res == true) {
                System.out.println("We found ");
            } else {
                System.out.println("We didnt
find");
            }
        }
    }

    static boolean find(int[] arr, int index, int
target, ArrayList<Integer> list) {

        if (target == 0) {
            System.out.println(list);
            return true;
        }
    }
}

```

```

        if (target < 0) {
            return false;
        }

        if (index == arr.length) {
            return false;
        }

        list.add(arr[index]);
        boolean left = find(arr, index + 1, target
- arr[index], list);
        list.remove(list.size() - 1);

        boolean right = find(arr, index + 1,
target, list);

        return left || right;
    }
}

```

34.

```

package Session_26;

public class CoinSumOpt {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 2, 5 };
        int target = 50;
        int[] memory = new int[target + 1];
        System.out.println(find(arr, target,
memory));
    }
}

```

```

    static int find(int[] arr, int target, int[]
memory) {

        if (target == 0) {
            return 1;
        }

        if (target < 0) {
            return 0;
        }

        if (memory[target] != 0) {
            return memory[target];
        }

        int count = 0;
        for (int i = 0; i < arr.length; i++) {
            count = count + find(arr, target -
arr[i], memory);
        }

        memory[target] = count;

        return count;
    }
}

```

35.

```
package Sessiopn_23;
```

```
import java.util.Stack;
```

```
public class FindValidParantius {
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
    }
}

```

```

String s = "()()[]([])";
find(s);
}

static void find(String s) {

    Stack<Character> statck = new Stack<>();

    for (int i = 0; i < s.length(); i++) {

        char c = s.charAt(i);

        if (c == '(') {
            statck.push(')');
        } else if (c == '[') {
            statck.push(']');
        }

        else {

            if(c!=statck.pop()) {
                System.out.println("Not
valid");
                return;
            }

        }

    }

    System.out.println("Valid");

}

}

```

36.

```
package Sessiopn_23;
```

```
public class FindValidString {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub
```

```
        String s = "()()()(";  
        find(s);
```

```
    }
```

```
    static void find(String s) {  
        int count = 0;
```

```
        for (int i = 0; i < s.length(); i++) {  
            if(s.charAt(i)=='(') {  
                count++;  
            }  
            else {  
                count--;  
            }  
        }
```

```
        System.out.println(count==0?"Valid": "Not  
Valid");
```

```
    }
```

```
}
```

37.

```
package StringBackTracing;
```

```

public class SubSequence {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String s = "abc";
        // String[] memory = new String[s.length()
+ 1];
        printAllSubsequences(s, 0, "");

    }

    static void printAllSubsequences(String str,
int index, String current) {

        if (str.isEmpty()) {
            System.out.println(current);
            return;
        }

        // Include the first character
        printAllSubsequences(str.substring(1),
index + 1, current + str.charAt(0));

        // Exclude the first character
        printAllSubsequences(str.substring(1),
index + 1, current);

    }

}

```

38.

```

package StringBackTracing;

import java.util.ArrayList;

public class SubStrings {

```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        String s = "abcd";
        find(s, 0, new ArrayList<>());
    }

    static void find(String s, int index,
ArrayList<Character> list) {

        if (index == s.length()) {
            return;
        }

        for (int i = index; i < s.length(); i++) {
            System.out.println(s.substring(index,
i + 1));
        }

        find(s, index + 1, list);
    }
}

```

39.

```

public class AddMatrices {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[][] mat1 = { { 1, 2 }, { 3, 4 }, { 5,
6 } };
        int[][] mat2 = { { 5, 6 }, { 7, 8 } };
    }
}

```



```

        int[][] result = find(mat1, mat2);
        for (int i = 0; i < result.length; i++) {
            for (int j = 0; j < result[0].length;
j++) {
                System.out.print(result[i][j] + "
");
            }
            System.out.println();
        }
    }

    static int[][] find(int[][] mat1, int[][]
mat2) {

        int row1 = mat1.length;
        int col1 = mat1[0].length;

        int row2 = mat2.length;
        int col2 = mat2[0].length;

        if (row1 != row2 || col1 != col2) {
            return null;
        }

        int[][] result = new int[row1][col2];

        for (int i = 0; i < row1; i++) {
            for (int j = 0; j < col1; j++) {
                result[i][j] = mat1[i][j] + mat2[i]
[j];
            }
        }
    }
}

```

```

        return result;

    }

}

40.

public class ArraySearch {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[][] matrix = { { 1, 2, 3 }, { 4, 5,
6 }, { 7, 8, 9 } };
        int target = 5;
        int rows = matrix.length;
        int cols = matrix[0].length;

        int row = 0;
        int col = cols - 1;

        while (row < rows && col >= 0) {
            if (matrix[row][col] == target) {
                System.out.println("Found");
                return;
            } else if (matrix[row][col] > target)
{
                col--; // Move left
            } else {
                row++; // Move down
            }
        }
        System.out.println("not found");
    }

}

```

42.

```
public class ArrayToSpiral {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] a = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
        int row = 3;
        int col = 3;
        int[][] result = new int[row][col];

        int leftcol = 0;
        int toprow = 0;
        int rightcol = col - 1;
        int bottomrow = row - 1;
        int arrayIndex = 0;
        int count = 0;
        int inputLen = a.length;
        while (leftcol <= rightcol && toprow <=
bottomrow) {
            if (toprow <= bottomrow) {

                for (int i = leftcol; i <=
rightcol; i++) {
                    count++;
                    if (count <= inputLen) {
                        result[toprow][i] =
a[arrayIndex];
                        arrayIndex++;
                    } else {
                        result[toprow][i] = -1;
                    }
                }
            }
            toprow++;
            if (leftcol <= rightcol) {
```

```

        for (int i = toprow; i <=
bottomrow; i++) {
            count++;
            if (count <= inputLen) {
                result[i][rightcol] =
a[arrayIndex++];
            } else {
                result[i][rightcol] = -1;
            }
        }
        rightcol--;
        if (bottomrow >= toprow) {
            for (int i = rightcol; i >=
leftcol; i--) {
                count++;
                if (count <= inputLen) {
                    result[bottomrow][i] =
a[arrayIndex++];
                } else {
                    result[bottomrow][i] = -1;
                }
            }
        }

        bottomrow--;
        if (leftcol <= rightcol) {
            for (int i = bottomrow; i >=
toprow; i--) {
                count++;
                if (count <= inputLen) {
                    result[i][leftcol] =
a[arrayIndex++];
                } else {
                    result[i][leftcol] = -1;
                }
            }
        }
    }
}

```

```

        leftcol++;
    }
    for (int i = 0; i < result.length; i++) {
        for (int j = 0; j < result[0].length;
j++) {
            System.out.print(result[i][j] + "
");
        }
        System.out.println();
    }
}
}

```

43.

```

import java.util.Arrays;

public class BetterOptimization {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[][] matrix = { { 1, 2, 3 }, { 4, 0,
5 }, { 6, 7, 8 } };
        find(matrix);
        for (int i = 0; i < matrix.length; i++) {

System.out.println(Arrays.toString(matrix[i]));

        }

    }

    static void find(int[][] matrix) {

        int row = matrix.length;

```

```

int col = matrix[0].length;

for (int i = 0; i < row; i++) {
    for (int j = 0; j < col; j++) {

        if (matrix[i][j] == 0) {
            matrix[0][j] = -1;
            matrix[j][0] = -1;
        }

    }

}

for (int i = 1; i < row; i++) {
    for (int j = 1; j < col; j++) {

        if (matrix[0][i] == -1 || matrix[j]
[0] == -1) {
            matrix[i][j] = 0;
        }

    }

}

for (int i = 0; i < row; i++) {
    if (matrix[0][i] == -1) {
        matrix[0][i] = 0;
    }

    if (matrix[i][0] == -1) {
        matrix[i][0] = 0;
    }

}

```

```

    }

}

44.
import java.util.Arrays;

public class CanAttendAllMeetings {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[][] intervals = { { 0, 30 }, { 30,
40 } };
        System.out.println(find(intervals));

    }

    static boolean find(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] -
b[0]);
        int previousStart = intervals[0][0];
        int previousEnd = intervals[0][1];

        for (int i = 1; i < intervals.length; i++)
        {
            int currentStart = intervals[i][0];
            if (currentStart < previousEnd) {
                return false;
            }
            previousStart = currentStart;

        }

        return true;
    }

}

```

45.

```
import java.util.Arrays;

public class ChangeValues {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        // arr = {9,12,3,5,14,10,10}

        int[] arr = { 9, 12, 3, 5, 14, 10, 10 };

        int lessIndex = 0;
        int equalIndex = 0;
        int largerIndex = arr.length - 1;

        while (equalIndex <= largerIndex) {
            int curr = arr[equalIndex];
            if (curr < 10) {
                swap(arr, lessIndex, equalIndex);
                lessIndex++;
                equalIndex++;
            } else if (curr == 10) {
                equalIndex++;
            } else {
                swap(arr, largerIndex, equalIndex);
                largerIndex--;
            }
        }

        System.out.println(Arrays.toString(arr));

    }

    static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
    }
}
```



```
        arr[j] = temp;

    }

}
```

46.

```
public class FactRectursion {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int n = 0;
        System.out.println(fact(n));
    }

    static int fact(int n) {
        if (n == 0) {
            return 1;
        }
        if (n == 1) {
            return 1;
        }

        return n * fact(n - 1);
    }

}
```

47.

```
public class FindDuplicateInArray {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 4, 4, 3, 2 };
    }

}
```

```

    int tortoise = arr[0];
    int hare = arr[0];

    do {
        tortoise = arr[tortoise];
        hare = arr[arr[hare]];
    } while (tortoise != hare);

    tortoise = arr[0];

    while (tortoise != hare) {

        tortoise = arr[tortoise];
        hare = arr[hare];
    }

    System.out.println(tortoise);
}

}

```

48.

```

public class FindFibonnaci {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int n = 6;
        System.out.println(fib(n));

    }

    static int fib(int n) {
        if (n == 1 || n == 0) {
            return n;
        }
    }
}

```

```

        return fib(n - 1) + fib(n - 2);
    }

}

49.
import java.util.Arrays;

public class FindLeftSideMaxOfEachElement {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = {3,4,2,1,3,6,7};
        find(arr);

    }

    static void find(int[] arr) {
        int len = arr.length;
        int[] result = new int[len];
        result[0] = arr[0];
        for (int i = 1; i < len; i++) {
            result[i] = Math.max(result[i - 1],
arr[i]);
        }

        System.out.println(Arrays.toString(result));

    }

}

```

```

50.

public class FindMaxPathSum {

```

```

static int max = Integer.MIN_VALUE;

public static void main(String[] args) {
    // TODO Auto-generated method stub

    int[][] mat = { { 1, 2, 3 },
                    { 4, 5, 6 },
                    { 7, 8, 9 } };

    int rows = mat.length;
    int cols = mat[0].length;
    int i = 0;
    int j = 0;
    int sum = mat[i][j];
    find(mat, rows, cols, i, j, sum);
    System.out.println(max);
}

static void find(int[][] mat, int rows, int
cols, int i, int j, int sum) {

    if (i == rows - 1 && j == cols - 1) {

        if (sum > max) {
            max = sum;
        }

    }

    if (i >= rows || j >= cols) {
        return;
    }

    if (j < cols - 1) {
        find(mat, rows, cols, i, j + 1, sum +
mat[i][j + 1]);
    }
}

```

```

        }

        if (i < rows - 1) {
            find(mat, rows, cols, i + 1, j, sum +
mat[i + 1][j]);
        }

    }

}

```

51.

```

public class FindThePeaak {

    public static void main(String[] args) {
        int[] nums1 = { 1, 2, 3, 1 };
        int index = findpeak(nums1);
        System.out.println(index);

        index = findPeakElement(nums1);

        System.out.println(index);
    }

    static int findpeak(int[] arr) {

        for (int i = 1; i < arr.length - 1; i++) {
            if (arr[i] > arr[i - 1] && arr[i] >
arr[i + 1]) {
                return i;
            }
        }

        return -1;
    }
}

```

```

    public static int findPeakElement(int[] nums)
    {
        int left = 0;
        int right = nums.length - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] > nums[mid - 1] &&
nums[mid] > nums[mid + 1]) {
                return mid;
            }

            if (nums[mid] > nums[mid + 1]) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return -1;
    }
}

```

52.

```

public class FirstMissingPositiveInteger {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 2, 3, 4, 5, 6, 7, 9 };
        int val = find(arr);

        System.out.println(val);
    }

    static int find(int[] arr) {

```

```
int[] numberLine = new int[10];
```

```
for (int i = 0; i < arr.length; i++) {  
    if (arr[i] < 0) {  
        arr[i] = 0;  
    }
```

```
    if (arr[i] > 9) {  
        arr[i] = 0;  
    }
```

```
}
```

```
for (int i = 0; i < arr.length; i++) {  
    int val = arr[i];  
    numberLine[val] = 1;
```

```
}
```

```
+) {  
    for (int i = 1; i < numberLine.length; i++) {  
        if (numberLine[i] == 0) {  
            return i;  
        }
```

```
}
```

```
return 0;
```

```
}
```

```
}
```

```

import java.util.HashMap;
import java.util.Map;

public class IsomorphicStringsExample {

    public static void main(String[] args) {
        String s1 = "bar";
        String s2 = "foo";

        if (areIsomorphic(s1, s2)) {
            System.out.println(s1 + " and " + s2 +
" are isomorphic.");
        } else {
            System.out.println(s1 + " and " + s2 +
" are not isomorphic.");
        }
    }

    private static boolean areIsomorphic(String
s1, String s2) {
        if (s1 == null || s2 == null ||
s1.length() != s2.length()) {
            return false;
        }

        Map<Character, Character> mapping = new
HashMap<>();

        for (int i = 0; i < s1.length(); i++) {
            char char1 = s1.charAt(i);
            char char2 = s2.charAt(i);

            // Check if char1 is already mapped to
a different character in s2
            if (mapping.containsKey(char1) &&
mapping.get(char1) != char2) {
                return false;
            }
        }
    }
}

```



```

        // Check if char2 is already mapped to
a different character in s1
        if (mapping.containsValue(char2) && !
mapping.containsKey(char1)) {
            return false;
        }

        // Establish the mapping
mapping.put(char1, char2);
    }

    return true;
}
}

```

54.

```

public class MatrixZigZag2 {

    public static void main(String[] args) {

        int[][] a = { { 1, 2, 3 }, { 4, 5, 6 },
{ 7, 8, 9 } };

        int rows = a.length;
        int cols = a[0].length;
        int startRow = 0;
        int startCol = 0;

        int timesoftraverse = rows + cols - 1;
        for (int k = 0; k < timesoftraverse; k++)
        {
            if (k % 2 == 0) {
                startRow = Math.min(k, rows - 1);
                startCol = Math.max(0, k - rows +
1);
            } else {

```

```

        startRow = Math.max(0, k - cols +
1);
        startCol = Math.min(k, cols - 1);
    }

    while (startRow >= 0 && startCol >= 0
&& startRow < rows && startCol < cols) {
        System.out.print(a[startRow]
[startCol] + " ");

        if (k % 2 == 0) {
            startRow--;
            startCol++;
        } else {
            startRow++;
            startCol--;
        }
    }

}

}
}

```

55.

```

import java.util.Arrays;

public class MaxConsecutiveElement {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 7, 8, 6, 3, 2, 4, 10,
13 };
        System.out.println(find(arr));

    }
}

```

```

static int find(int[] arr) {
    int max = Integer.MIN_VALUE;

    for (int i = 0; i < arr.length; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }

    int[] numberLine = new int[max + 2];

    for (int i = 0; i < arr.length; i++) {
        numberLine[arr[i]] = 1;
    }

    int count = 0;
    int maxCount = 0;

```

```

System.out.println(Arrays.toString(numberLine));

```

```

    for (int i = 0; i < numberLine.length; i++) {

        if (numberLine[i] == 1) {
            count++;
            if (count > maxCount) {
                maxCount = count;
            }
            while (numberLine[i] != 0) {
                if (numberLine[i] ==
numberLine[i + 1]) {
                    i++;
                    count++;
                    if (count > maxCount) {
                        maxCount = count;
                    }
                } else {

```

```

        count = 0;
        break;
    }

}

}

}

return maxCount;

}

}

```

56.

```

public class MaxNonRepeteadSubString {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String ip = "abbcdb";
        int[] arr = new int[26];
        int len = ip.length();

        int start = 0;
        int left = 0;
        String sub = "";
        for (int i = 0; i < len; i++) {
            int c = arr[ip.charAt(i) - 97];
            if (c > start) {
                start = c;
                left = i;
            }
            arr[ip.charAt(i) - 97]++;
        }
    }
}

```

```

        if (sub.length() < ip.substring(left,
i + 1).length()) {
            sub = ip.substring(left, i + 1);
        }

    }
    System.out.println(sub);

}

}

```

57.

```

import java.util.Arrays;

public class MinNoOfClassRoomsRequired {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[][] classTimings = { { 1, 3 }, { 2,
4 }, { 3, 7 }, { 6, 8 } };
        // { { 30, 75 }, { 0, 50 }, { 60, 150 } };
        // { { 1, 3 }, { 2, 4 }, { 3, 7 }, { 6,
8 } };
        int val = find(classTimings);
        System.out.println(val);

    }

    static int find(int[][] intervals) {

        int len = intervals.length;

        Arrays.sort(intervals, (a, b) -> a[0] -
b[0]);

        for (int i = 0; i < intervals.length; i++)
        {

```

```

System.out.println(Arrays.toString(intervals[i]));

    }

    int numLine[] = new int[10];

    for (int i = 0; i < intervals.length; i++)
    {
        int start = intervals[i][0];
        int end = intervals[i][1];
        if (numLine[start] != 2)
            numLine[start] = 1;
        numLine[end] = 2;
    }

    System.out.println(Arrays.toString(numLine));
    // 0 1 1 2 2 0 1 2 2 0 0

    int count = 0;
    int free = 0;

    for (int i = 0; i < numLine.length; i++) {
        if (numLine[i] == 1) {
            if (free == 0) {
                count++;
            } else
                free--;
        }
        if (numLine[i] == 2) {
            if (count > free)
                free++;
        }
    }
}

```

```

        return count;

    }

}

55.

public class MobileCombinations {

    static String[] combination = new String[]
{ "abc", "def", "ghi", "jkl", "mno", "pqr", "stu",
"vwx", "yz" };

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        String ip = "12";
        find("", ip, 0);

    }

    static void find(String s, String ipString,
int index) {

        if (s.length() == ipString.length()) {
            System.out.println(s);
            return;
        }

        int digit = ipString.charAt(index) - '1';
        String currentString = combination[digit];
        char[] chararr =
currentString.toCharArray();
        for (int i = 0; i < chararr.length; i++) {
            find(s + chararr[i], ipString, index +
1);
        }
    }
}

```

```
}
```

```
}
```

56.

```
public class MostFrequentEvenNum {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
  
        int input[] = { 29, 47, 21, 41, 13, 37,  
25, 7 };  
        int arr[] = new int[48];  
        int index = -1;  
        int mostFreq = 0;  
        int currentEleFreq = 0;  
  
        for (int i = 0; i < input.length; i++) {  
            arr[input[i]]++;  
        }  
  
        for (int i = arr.length - 1; i >= 0; i--)  
{  
  
            if (i % 2 == 0 && arr[i] > 0) {  
  
                currentEleFreq = arr[i];  
                if (mostFreq == 0) {  
                    mostFreq = currentEleFreq;  
                    index = i;  
                } else if (currentEleFreq >=  
mostFreq) {  
                    mostFreq = currentEleFreq;  
                    index = i;  
                }  
            }  
        }  
    }  
}
```



```

        }

    }

    System.out.println(index);

}

}

```

57.

```

import java.util.Arrays;

public class MulArray {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[][] m1 = { { 1, 2 }, { 3, 4 } ,
{3,4}};
        int[][] m2 = { { 5, 6, 7 }, { 7, 8 ,10} ,
{11,23,45}};
        int[][] result = mul(m1, m2);
        for (int i = 0; i < result.length; i++) {

System.out.println(Arrays.toString(result[i]));

        }

    }

    static int[][] mul(int[][] m1, int[][] m2) {

        int row1 = m1.length;
        int col1 = m1[0].length;

        int row2 = m2.length;

```

```

        int col2 = m2[0].length;

        if (col1 != row2) {
            return null;
        }

        int[][] result = new int[row1][col2];

        for (int i = 0; i < row1; i++) {
            for (int j = 0; j < col2; j++) {
                for (int k = 0; k < row2; k++) {
                    result[i][j] = result[i][j] +
m1[i][k] * m2[k][j];
                }
            }
        }

        return result;
    }
}

```

58.

```

public class NoOfPossibleWayToreachPosition {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int n = 3;
        System.out.println(find(n, 0));
    }
}

```

```
}
```

```
static int find(int n, int stpeCount) {  
    if (stpeCount == n) {  
        return 1;  
    }  
    if (stpeCount > n) {  
        return 0;  
    }  
  
    int onecount = find(n, stpeCount + 1);  
    int twocount = find(n, stpeCount + 2);  
  
    return onecount + twocount;  
}
```

```
}
```

59.

```
import java.util.ArrayList;  
import java.util.List;
```

```
public class PrintAllPaths {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub
```

```
        Node root = new Node(1);  
        root.left = new Node(2);  
        root.right = new Node(3);
```

```
        root.left.left = new Node(4);  
        root.left.right = new Node(5);
```

```
        root.right.left = new Node(6);  
        root.right.right = new Node(7);
```

```

        System.out.println(findPath(root));
    }

    static List<List<Integer>> findPath(Node root)
    {
        List<List<Integer>> allPaths = new
ArrayList<>();
        List<Integer> currentPath = new
ArrayList<>();

        dfs(root, currentPath, allPaths);

        return allPaths;
    }

    static void dfs(Node root, List<Integer>
currentPath, List<List<Integer>> allPaths) {
        if (root == null) {
            return;
        }
        currentPath.add(root.val);
        if (root.left == null && root.right ==
null) {
            allPaths.add(new
ArrayList<>(currentPath));
        } else {
            dfs(root.left, currentPath, allPaths);
            dfs(root.right, currentPath,
allPaths);
        }
        currentPath.remove(currentPath.size() -
1);
    }
}

```

```

class Node {
    int val;
    Node left;
    Node right;

    Node(int val) {
        this.val = val;
    }
}

```

60.

```

import java.util.ArrayList;
import java.util.HashMap;

public class PrintPossibleAlphabetCombinations {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        String s = "1221";
        HashMap<Integer, Character> map = new
HashMap<>();

        for (char i = 'a'; i <= 'z'; i++) {
            map.put(i - 'a' + 1, i);
        }
        System.out.println(map);
        ArrayList<String> list = new
ArrayList<>();

        System.out.println(find(s, 0, list, map));

    }

    static int find(String s, int index,
ArrayList<String> list, HashMap<Integer,
Character> map) {

```

```

        if (index == s.length()) {
            System.out.println(list);
            return 1;
        }

        if (index > s.length()) {
            return 0;
        }

        if (s.charAt(index) == '0') {
            return 0;
        }

        int val = 0;

        int digit = s.charAt(index) - '0';
        char c = map.get(digit);
        list.add("" + c);
        val = find(s, index + 1, list, map);
        list.remove("" + c);

        if (index < s.length() - 1 &&
Integer.parseInt(s.substring(index, index + 2)) <=
26) {
            String str1 = s.substring(index, index
+ 2);
            int num = Integer.parseInt(str1);
            char chars = map.get(num);

            list.add("" + chars);
            val = val + find(s, index + 2, list,
map);
            list.remove("" + chars);
        }

        return val;
    }

```

```
}
```

62.

```
public class PrintValidPerantisis {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
  
        int n = 10;  
        find(n, 0, 0, "");  
  
    }  
  
    static void find(int n, int open, int close,  
String s) {  
  
        if (s.length() == 2 * n) {  
            System.out.println(s);  
            return;  
        }  
  
        if (open < n) {  
            find(n, open + 1, close, s + "(");  
        }  
  
        if (close < open) {  
            find(n, open, close + 1, s + ")");  
        }  
  
    }  
  
}
```

63.

```
public class SumRecursion {
```

```

public static void main(String[] args) {
    // TODO Auto-generated method stub

    int n = 123;
    System.out.println(find(n));
}

static int find(int n) {
    if (n == 0) {
        return 0;
    }
    return n % 10 + find(n / 10);
}
}

```

64.

```

public class TopLeftToBottomRight {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[][] mat = { { 0, 0 }, { 0, 0 } };

        int rows = mat.length;
        int col = mat[0].length;
        int i = 0;
        int j = 0;

        int ways = find(mat, rows, col, i, j);
        System.out.println(ways);
    }
}

```



```

static int find(int[][] mat, int rows, int
cols, int i, int j) {

    if (i == rows - 1 && j == cols - 1) {
        return 1;
    }
    if (i >= rows || j >= cols) {
        return 0;
    }

    int rightval = find(mat, rows, cols, i, j
+ 1);

    int leftval = find(mat, rows, cols, i + 1,
j);

    return rightval + leftval;

}

}

```

65.

```

import java.util.Arrays;

public class WiggleSort {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 2, 3, 4, 1 };
        for (int i = 0; i < arr.length - 1; i++) {
            if ((i % 2 == 0 && arr[i] > arr[i +
1]) || (i % 2 == 1 && arr[i] < arr[i + 1])) {
                swap(arr, i, i + 1);
            }
        }
    }
}

```

```

        System.out.println(Arrays.toString(arr));
    }

    static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

```

66.

```

package ArraysMissing;

import java.util.ArrayList;

public class ProductOfSubArrayLessthenK {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 10, 5, 2, 6 };
        int k = 100;
        System.out.println(find(arr, k));
    }

    static int find(int[] arr, int k) {
        int count = 0;
        ArrayList<Integer> list = new
ArrayList<>();
        int start = 0;
        int mul = 1;
        for (int i = 0; i < arr.length; i++) {
            mul = mul * arr[i];
            list.add(arr[i]);
            while (mul >= k) {
                mul = mul / arr[start];
            }
        }
    }
}

```

```

        list.remove(start);
        start++;
    }
    count = count + i - start + 1;
    System.out.println(list);
}

return count;
}
}

```

67

```
package Level_2;
```

```
public class Main {
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 2, 3, 5, 8, 9, 10, 14,
15, 16, 17, 18, 19, 20 };
        int val = 20;
        search(arr, val);
        binarySearch(arr, val);
    }

```

```

static void search(int[] arr, int val) {
    int count = 0;

    // O(n)
    for (int i = 0; i < arr.length; i++) {
        count++;

        if (arr[i] == val) {
            System.out.println("Value found");
            return;
        }
    }
}

```

```

        }
        System.out.println("Count : " +
count);
    }
    System.out.println("Value is not found");
}

static void binarySearch(int[] arr, int val) {

    int start = 0;
    int end = arr.length - 1;
    int count = 0;

    // O(log(n))

    while (start <= end) {
        count++;
        System.out.println("Running loop " +
count);
        int mid = (end + start) / 2;
        if (arr[mid] > val) {
            end = mid - 1;
        } else if (arr[mid] < val) {
            start = mid + 1;
        } else {
            System.out.println("Value found");
            return;
        }
    }

}

}

```

68.
package Level_2;

```

// Reverse only vowels
// ip--> hello
// op--> holle
public class Q1 {
    static int count = 0;

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        String input = "Zoho corporation
adiuyirisf adenjejoirwq799r7qub rtebuuithuiqt trtt
te";

        System.out.println(input.length());
        int start = 0;
        int end = input.length() - 1;

        char[] chararray = input.toCharArray();
        changeVowelPositions(chararray, start,
end);

        String result = new String(chararray);
        System.out.println(result);
        System.out.println(count);

    }

    static void changeVowelPositions(char[]
chararray, int start, int end) {
        while (start < end) {
            count++;

            while (start < end && !
isVowel(chararray[start])) {
                start++;
                count++;

            }

```

```

        while (start < end && !
isVowel(chararray[end])) {
            end--;
            count++;

        }

        if (start < end) {
            swap(chararray, start, end);
            start++;
            end--;

        }

    }

//      while(start<end){
//          count++;
//          if(!isvowel(s[start])) start++;
//          else if(!isvowel(s[end])) end--;
//          else if(isvowel(s[start]) ==
isvowel(s[end])){
//              char temp=s[start];
//              s[start++]=s[end];
//              s[end--]=temp;
//          }
//      }
//  }

static void swap(char[] chararray, int start,
int end) {
    char temp = chararray[start];
    chararray[start] = chararray[end];
    chararray[end] = temp;
}

static boolean isVowel(char c) {
    return "aeiouAEIOU".indexOf(c) != -1;
}

```

```
}
```

```
}
```

69.

```
package Level_2;
```

```
import java.util.Arrays;
```

```
// Median a=of two sorted arrays when merge into  
one
```

```
public class Q3 {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub
```

```
        int[] first = { 1, 2, 3  };
```

```
        int[] second = { 4, 5, 6 };
```

```
        int i = first.length;
```

```
        int j = second.length;
```

```
        int[] mergearray = new int[i + j];
```

```
        i = 0;
```

```
        j = 0;
```

```
        int k = 0;
```

```
        while (i < first.length && j <  
second.length) {  
            if (first[i] < second[j]) {  
                mergearray[k++] = first[i++];  
            } else {  
                mergearray[k++] = second[j++];  
            }  
        }
```

```

    }

    while (i < first.length) {
        mergearray[k++] = first[i++];
    }

    while (j < second.length) {
        mergearray[k++] = second[j++];
    }

```

```

System.out.println(Arrays.toString(mergearray));

```

```

    if (mergearray.length % 2 == 1) {

```

```

System.out.println(mergearray[mergearray.length /
2]);

```

```

    } else {

```

```

        System.out

```

```

            .println((double)

```

```

(mergearray[mergearray.length / 2 - 1] +
mergearray[mergearray.length / 2]) / 2);

```

```

    }

```

```

}

```

```

}

```

```

70.

```

```

import java.util.HashSet;

```

```

import java.util.Set;

```

```

public class Leetcode_1_TwoSum {

```

```

    public static void main(String[] args) {

```

```

        // TODO Auto-generated method stub

```

```

        int[] ip = { 4, 5, 2, 1, 3 };

```



```

        int target = 6;
        int[] result = findSum(ip, target);
        System.out.println(result[0] + " " +
result[1]);

    }

    static int[] findSum(int[] arr, int target) {
        int len = arr.length;
        int[] op = new int[2];
        Set<Integer> set = new HashSet();
        for (int i = 0; i < len; i++) {
            int remining = target - arr[i];
            if (set.contains(arr[i])) {
                return new int[] { arr[i], remining
};
            }
            set.add(remining);
        }
        return op;
    }
}

```

71.

```

package Strings;

import java.util.ArrayList;
import java.util.Arrays;

public class EncodeDecodeString {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        ArrayList<String> list = new
ArrayList<>();
    }
}

```

```

        list.add("Hello");
        list.add("World");
        list.add(":");
        list.add("C");

        String encoded = encode(list);

        System.out.println(encoded);

        System.out.println(decode(encoded));
    }

    static String encode(ArrayList<String> list) {

        int len = list.size();
        String WildChar = "#";
        String op = "";

        for (int i = 0; i < len; i++) {
            String word = list.get(i);
            op = op + word + WildChar;
        }

        return op;
    }

    static ArrayList<String> decode(String s) {

        ArrayList<String> list = new ArrayList();
        String[] val = s.split("#");
        for (int i = 0; i < val.length; i++) {
            list.add(val[i]);
        }

        return list;
    }

```

```
}
```

```
}
```

72.

```
package Strings;
```

```
import java.util.HashMap;
```

```
/**
```

```
 * https://leetcode.com/problems/isomorphic-strings/description/ Given two
```

```
 * strings s and t, determine if they are isomorphic.
```

```
 *
```

```
 * Two strings s and t are isomorphic if the characters in s can be replaced to
```

```
 * get t.
```

```
 *
```

```
 * All occurrences of a character must be replaced with another character while
```

```
 * preserving the order of characters. No two characters may map to the same
```

```
 * character, but a character may map to itself.
```

```
 *
```

```
 *
```

```
 *
```

```
 * Example 1:
```

```
 *
```

```
 * Input: s = "egg", t = "add" Output: true
```

```
Example 2:
```

```
 *
```

```
 * Input: s = "foo", t = "bar" Output: false
```

```
Example 3:
```

```
 *
```

```
 * Input: s = "paper", t = "title" Output: true
```

```
 *
```

```
 */
```

```

public class IsomorphicStrings {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String s1 = "egg";
        String s2 = "add";
        System.out.println(isIsomorphic(s1, s2));
    }

    static boolean isIsomorphic(String s1, String
s2) {
        HashMap<Character, Character> map = new
HashMap<>();
        if (s1.length() != s2.length()) {
            return false;
        }

        for (int i = 0; i < s1.length(); i++) {
            if (map.containsKey(s1.charAt(i))) {
                char c = map.get(s1.charAt(i));
                if (c != s2.charAt(i)) {
                    return false;
                }
            } else {
                map.put(s1.charAt(i),
s2.charAt(i));
            }
        }

        return true;
    }
}

```

73.

```
package arrays;
```

```
public class AnagramArray {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub
```

```
        int[] arr1 = { 1, 2, 3, 2, 1, 5 }; // 1- 2  
        , 2- 2, 3-1
```

```
        int[] arr2 = { 2, 3, 2, 1, 3 }; // 1- 1,  
        2- 2, 3-2
```

```
        System.out.println(check(arr1, arr2));
```

```
    }
```

```
    static boolean check(int[] arr1, int[] arr2) {
```

```
        int[] freq = new int[10];
```

```
        if (arr1.length != arr2.length) {  
            return false;  
        }
```

```
        for (int i = 0; i < arr1.length; i++) {  
            int val = arr1[i];  
            freq[val]++;  
        }
```

```
        for (int i = 0; i < arr2.length; i++) {  
            int val = arr2[i];  
            freq[val]--;  
        }
```

```
        for (int i = 0; i < freq.length; i++) {  
            if (freq[i] != 0) {  
                return false;  
            }  
        }
```

```
    }  
  
    return true;  
}  
  
}
```

74.

```
package arrays;
```

```
public class BestTimeToBuyAndSell {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        int[] arr = { 7, 1, 5, 3, 6, 4 };  
        int profit = maxProfit(arr);  
        System.out.println(profit);  
    }  
  
    static int maxProfit(int[] arr) {  
        int buy = arr[0];  
        int sell = arr[1];  
        int maxProfit = Integer.MIN_VALUE;  
        int currentProfit = 0;  
  
        for (int i = 1; i < arr.length; i++) {  
  
            currentProfit = arr[i] - buy;  
            if (currentProfit > maxProfit) {  
                maxProfit = currentProfit;  
                sell = arr[i];  
            }  
  
            if (arr[i] <= buy) {  
                buy = arr[i];  
            }  
        }  
    }  
}
```

```

        }

        return maxProfit;
    }
}

75.
package arrays;

public class BinarySearchInRecursiveWay {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 2, 3, 4, 5, 6, 7, 8 };
        int target = 20;
        int start = 0;
        int end = arr.length - 1;

        int index = binarySearch(arr, start, end,
target);
        System.out.println(index);
    }

    static int binarySearch(int[] arr, int start,
int end, int target) {

        int mid = (start + end) / 2;

        if (start > end) {
            return -1;
        }

        if (arr[mid] == target) {

```

```

        return mid;
    }

    if (arr[mid] > target) {

        return binarySearch(arr, start, mid -
1, target);

    }

    return binarySearch(arr, mid + 1, end,
target);

}

}

```

76.

```
package arrays;
```

```
import java.util.HashMap;
```

```
public class ContainsNearbyDuplicates {
```

```

    public static void main(String[] args) {
        int[] arr = { 1,2,3,1};
        int window = 3;
        boolean val = containsNearbyDuplicate(arr,
window);
        System.out.println(val);
    }

```

```

    static boolean containsNearbyDuplicate(int[]
arr, int k) {
        HashMap<Integer, Integer> map = new
HashMap<Integer, Integer>();
        for (int i = 0; i < arr.length; i++) {
            if (!map.containsKey(arr[i])) {

```



```

        map.put(arr[i], i);
    } else {
        int val = map.get(arr[i]);
        System.out.println(val + " " + i);
        if (val - i <= k) {
            return true;
        }
        map.put(arr[i], i);
    }
}

return false;
}
}

```

77

```

package arrays;

import java.util.Arrays;

public class DivideAndConquer {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 10, 2, 3, 8, 12, 32, 7, 6,
3, 1 };
        mergeSort(arr);
        System.out.println(Arrays.toString(arr));
    }

    static void mergeSort(int[] arr) {

        int[] result = new int[arr.length];
        int start = 0;
    }
}

```

```

    int end = arr.length - 1;

    divide(arr, result, start, end);
}

static void divide(int[] arr, int[] result,
int start, int end) {

    if (start == end) {
        return;
    }

    int mid = (start + end) / 2;

    divide(arr, result, start, mid);
    divide(arr, result, mid + 1, end);

    mergeandsort(arr, result, start, mid,
end);
}

static void mergeandsort(int[] arr, int[]
result, int start, int mid, int end) {
    int i = start;
    int j = mid + 1;

    int k = start;

    while (i <= mid && j <= end) {

        if (arr[i] <= arr[j]) {
            result[k++] = arr[i++];
        } else {
            result[k++] = arr[j++];
        }
    }
}

```

```

    }

    while (i <= mid) {
        result[k++] = arr[i++];
    }

    while (j <= end) {
        result[k++] = arr[j++];
    }

    for (int index = start; index <= end;
index++) {

        arr[index] = result[index];

    }

}

}

```

78.

```
package arrays;
```

```

public class FindkMaxSumSubArray {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 2, 3, 4, 5, 6, -6 };
        int window = 3;
        System.out.println(max(arr, window));

    }

    static int max(int[] arr, int window) {

```

```

        int arr_len = arr.length;
        int maxSum = 0;
        int currentSum = 0;
        for (int i = 0; i < window; i++) {
            currentSum = currentSum + arr[i];
        }
        maxSum = currentSum;

        for (int i = window; i < arr_len; i++) {
            currentSum = currentSum + arr[i] -
arr[i - window];
            if (currentSum > maxSum) {
                maxSum = currentSum;
            }

        }

        return maxSum;
    }
}

```

79.

```
package arrays;
```

```
public class FindMedian {
```

```
    public static void main(String[] args) {
        // TODO Auto-generated method stub

```

```
        int[] arr1 = { 1, 2 };
        int[] arr2 = { 3, 4 };

```

```
        double result = median(arr1, arr2);
        System.out.println(result);

```

```
    }
```

```

static double median(int arr1[], int[] arr2) {
    int arr1_len = arr1.length;
    int arr2_len = arr2.length;

    int[] result = new int[arr1_len +
arr2_len];
    int i = 0;
    int j = 0;
    int k = 0;

    while (i < arr1_len && j < arr2_len) {
        if (arr1[i] <= arr2[j]) {
            result[k++] = arr1[i++];
        } else {
            result[k++] = arr2[j++];
        }
    }

    while (i < arr1_len) {
        result[k++] = arr1[i++];
    }

    while (j < arr2_len) {
        result[k++] = arr2[j++];
    }

    int resultArr_len = result.length;
    if (resultArr_len % 2 == 1) {
        return result[resultArr_len / 2];
    }

    return (result[resultArr_len / 2] +
result[(resultArr_len / 2) - 1]) / 2.0;
}

```

```
}
```

80.

```
package arrays;
```

```
public class FindSquareRoot {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        int num = 10;  
        int op = findSquare(num);  
        System.out.println(op);
```

```
    }
```

```
    static int findSquare(int num) {
```

```
        int start = 0;  
        int end = num;  
        int result = 0;
```

```
        while (start <= end) {  
            int mid = (start + end) / 2;  
            int mul = mid * mid;
```

```
            if (mul == num) {  
                return mid;  
            } else if (mul > num) {  
                end = mid - 1;
```

```
            } else {  
                start = mid + 1;  
                result = mid;
```

```
            }
```

```
        }
```

```
        return result;
```

```
}
```

```
}
```

81.

```
package arrays;
```

```
public class FindValueInRotatedSortedArray {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub
```

```
        int[] arr = { 4, 5, 6, 7, 8, 0, 1, 2, 3 };  
        // { 1, 2, 3, 4, 5, 6, 7, 3, 2, 1 };
```

```
        // System.out.println(arr[peak(arr)]);
```

```
        System.out.println(rotatedPeak(arr));
```

```
    }
```

```
    static int peak(int[] arr) {
```

```
        int start = 0;
```

```
        int end = arr.length - 1;
```

```
        int peak = 0;
```

```
        while (start <= end) {
```

```
            int mid = (start + end) / 2;
```

```
            if (arr[mid] > arr[mid + 1] &&
```

```
arr[mid] > arr[mid - 1]) {
```

```
                return mid;
```

```
            } else if (arr[mid + 1] > arr[mid]) {
```

```
                start = mid + 1;
```

```
            } else {
```

```
                end = mid - 1;
```

```
            }
```

```
        }
```

```
        return -1;
```

```

    }

    static int rotatedPeak(int[] nums) {
        int start = 0;
        int end = nums.length - 1;
        while (start <= end) {
            int mid = (start + end) / 2;

            // Check if mid is the maximum element
            if (nums[mid] > nums[mid + 1] &&
nums[mid] > nums[mid - 1]) {
                return nums[mid];
            }

            if (nums[mid] > nums[end]) {
                start = mid + 1;
            } else {
                end = mid - 1;
            }
        }
        return -1;
    }
}

```

82.

```
package arrays;
```

```

public class KadanesALgoForMaxSumOfSubArray {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 3, -1, -2, 4, 3, 7, -2 };
        int max = findMax(arr);
        System.out.println(max);
    }
}

```



```

    }

    static int findMax(int[] arr) {
        int currentSum = 0;
        int max = 0;

        for (int i = 0; i < arr.length; i++) {

            currentSum = Math.max(arr[i],
currentSum + arr[i]);
            max = Math.max(max, currentSum);

        }

        return max;
    }
}

```

83.

```
package arrays;
```

```
public class LinearSearch {
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int arr[] = { 10, 20, 30, 40, 50 };
        int target = 25;
        int start = 0;
        int end = arr.length - 1;
        int foundIndex = 0;

        // foundIndex = lsearch(arr, target);
        foundIndex = lsearchrecursiveways(arr,
target, start, end);

        if (foundIndex == -1) {

```

```

        System.out.println("Not found");
    } else {
        System.out.println("target found at "
+ foundIndex + " index");
    }
}

```

```

static int lsearch(int[] arr, int target) {

```

```

    int start = 0;
    int end = arr.length - 1;

```

```

    while (start < end) {
        int mid = (start + end) / 2;
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            start = mid + 1;
        } else {
            end = mid - 1;
        }
    }

```

```

    return -1;
}

```

```

static int lsearchrecursiveways(int[] arr, int
target, int start, int end) {
    if (start > end) {
        return -1;
    }

```

```

    int mid = (start + end) / 2;

```

```

    if (arr[mid] == target) {
        return mid;
    }

```

```

        if (arr[mid] < target) {
            return lsearchrecursiveways(arr,
target, mid + 1, end);
        }
        return lsearchrecursiveways(arr, target,
start, mid - 1);
    }
}

```

84.

```
package arrays;
```

```
public class MedianOfSortedArray {
```

```
    public static void main(String[] args) {
        // TODO Auto-generated method stub

```

```
        int[] arr1 = { 1, 2, 3 };
        int[] arr2 = { 4, 5, 6 };

```

```
    }
```

```
    static double median(int[] arr1, int[] arr2) {
```

```
        if (arr1.length < arr2.length) {
            return median(arr2, arr1);
        }

```

```
        int start1 = 0;
        int end1 = arr1.length - 1;

```

```
        int start2 = 0;
        int end2 = arr2.length - 1;

```

```
        while (start1 < end1) {
```

```

        int mid = (start1 + end1) / 2;

        if (arr1[mid] > arr2[end2]) {
            mid--;
        } else {
            end2--;
        }

    }

    return 0;
}
}

```

85.

```

package arrays;

import java.util.Arrays;

public class MoveZerosToRight {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 0, 4, 5, 3, 0, 0, 4, 0 };
        move(arr);
        System.out.println(Arrays.toString(arr));
    }

    static void move(int[] arr) {
        int start = 0;
        int end = arr.length - 1;
        while (start <= end) {
            if (arr[start] == 0) {
                swap(arr, start, end);
                end--;
            } else {

```

```

        start++;
    }

}

static void swap(int[] arr, int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}

}

```

86.

```
package arrays;
```

```

public class PeakIIInRotatedSortedArray {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        // 1,2,3,4,5,6,7 --> 3 times left rotated

        int[] arr = { 4, 5, 6, 7, 1, 2, 3 };
        int peak = find(arr);

        System.out.println(peak);
    }

    static int find(int[] arr) {
        int start = 0;
        int end = arr.length - 1;

        while (start <= end) {
            int mid = (start + end) / 2;

```

```

        if (arr[mid - 1] < arr[mid] &&
arr[mid] > arr[mid + 1]) {
            return arr[mid];
        }

        if (arr[mid] > arr[end]) {
            start = mid + 1;
        }

        else {
            end = mid - 1;
        }

    }

    return -1;
}
}

```

87.

```

package arrays;

import java.util.Arrays;

public class ReverseAnArray {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 2, 3, 4, 6 };
        reverse(arr);

        System.out.println(Arrays.toString(arr));
    }

    static void reverse(int[] arr) {

```

```

    int start = 0;
    int end = arr.length - 1;
    while (start <= end) {
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }
}

```

88.

```

package arrays;

import java.util.Arrays;

public class ReverseOnlyOddPositions {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 2, 3, 4, 6 };
        revreseOdd(arr);
        System.out.println(Arrays.toString(arr));
    }

    static void revreseOdd(int[] arr) {
        int start = 1;
        int end = arr.length - 1;

        while (start < end) {
            if (end % 2 != 1) {
                end--;
            }

            int temp = arr[start];

```

```

        arr[start] = arr[end];
        arr[end] = temp;
        start = start + 2;
        end = end - 2;
    }

```

```

    }

```

```

}

```

89.

```

package arrays;

```

```

import java.util.Arrays;

```

```

public class Segregate012s {

```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub

```

```

        int[] arr = { 1, 1, 0, 0, 2, 1, 0 };
        segregate(arr);
        System.out.println(Arrays.toString(arr));

```

```

    }

```

```

    static void segregate(int[] arr) {
        int zeroIndex = 0;
        int curentIndex = 0;
        int twoIndex = arr.length - 1;

```

```

        while (curentIndex <= twoIndex) {
            if (arr[curentIndex] == 0) {
                swap(arr, curentIndex, zeroIndex);
                zeroIndex++;
                curentIndex++;
            }

```



```

        } else if (arr[curentIndex] == 2) {
            swap(arr, curentIndex, twoIndex);
            twoIndex--;
        } else {
            curentIndex++;
        }
    }
}

```

```

static void swap(int[] arr, int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}

```

```

}

```

90.

```

package arrays;

```

```

import java.util.Arrays;

```

```

public class ShiftZerosRight {

```

```

    static void shiftZeros(int[] nums) {
        int nonZeroPointer = 0;

        // Move non-zero elements to the left
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                swap(nums, i, nonZeroPointer);
                nonZeroPointer++;
            }
        }
    }
}

```

```

// Helper method to swap elements in the array
static void swap(int[] nums, int i, int j) {
    int temp = nums[i];
    nums[i] = nums[j];
    nums[j] = temp;
}

public static void main(String[] args) {
    // Example usage
    int[] array = { 0, 2, 0, 3, 4, 0, 1, 5 };
    System.out.println("Before shifting zeros:
" + Arrays.toString(array));

    shiftZeros(array);

    System.out.println("After shifting zeros:
" + Arrays.toString(array));
}

```

91.

```
package arrays;
```

```
public class SlidingWindow {
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 2, 3, 4, 5, -5 };
        int windowSize = 3;

        int op = max(arr, windowSize);
        System.out.println(op);
    }

```

```

    static int max(int[] arr, int windowSize) {

        int max = 0;
    }

```

```

        int currenSum = 0;

        for (int i = 0; i < windowSize; i++) {
            currenSum = currenSum + arr[i];
        }

        max = currenSum;

        for (int i = windowSize; i < arr.length;
i++) {
            currenSum = currenSum + arr[i] - arr[i
- windowSize];

            if (currenSum > max) {
                max = currenSum;
            }
        }

        return max;
    }
}

```

92.

```

package arrays;

import java.util.Arrays;

public class Test12321 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        solution();
    }
}

```

```

    public static boolean checks(char i) {
        if (i == 'a' || i == 'e' || i == 'i' || i
== 'o' || i == 'u' || i == 'A' || i == 'E' || i ==
'I' || i == 'O'
            || i == 'U')
            return true;
        return false;
    }

```

```

    public static String solution() {
        String s = "hello";
        char[] arr = s.toCharArray();
        int i = 0, j = s.length();
        while (i < j) {
            System.out.println(" hello");
            if (checks(s.charAt(i)) &&
checks(s.charAt(i))) {
                swap(arr, i, j);

            } else if (checks(s.charAt(i)))
                j--;
            else if (checks(s.charAt(i)))
                i++;
            else {
                i++;
                j--;
            }
        }
        System.out.println(Arrays.toString(arr));

        return s;
    }

```

```

    static void swap(char[] c, int i, int j) {
        char temp = c[i];
        c[i] = c[j];
        c[j] = temp;
    }

```

```
    }  
}
```

93.

```
package arrays;
```

```
import java.util.Arrays;
```

```
public class TwoUnSortedArraysSortInaSingleArray {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub
```

```
        int[] arr1 = { 7, 9, 1, 3, 10 };
```

```
        int[] arr2 = { 5, 8, 22, 4 };
```

```
        int[] result = new int[arr1.length +  
arr2.length];
```

```
        mergeInSortedWay(arr1, arr2, result);
```

```
        System.out.println(Arrays.toString(result));
```

```
    }
```

```
    static void mergeInSortedWay(int[] arr1, int[]  
arr2, int[] result) {
```

```
        int i = 0;
```

```
        int j = 0;
```

```
        int k = 0;
```

```
        while (i < arr1.length && j < arr2.length)  
{
```

```
            if (arr1[i] < arr2[j]) {  
                result[k++] = arr1[i++];
```

```
            } else {  
                result[k++] = arr1[j++];
```

```
            }
```

```

    }
    while (i < arr1.length) {
        result[k++] = arr1[i++];
    }
    while (j < arr2.length) {
        result[k++] = arr1[j++];
    }
}

}

94.
package ArraysMissed;

import java.util.ArrayList;

public class CombinationOfAStringsByCallingNumber
{
    private static final String[] LETTERS =
{ "abc", // 1
    "def", // 2
    "ghi", // 3
    "jkl", // 4
    "mno", // 5
    "pqr", // 6
    "stu", // 7
    "vwx", // 8
    "yz" // 9
};

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String digits = "123";

        System.out.println(find("", digits, 0));
    }
}

```

```

    static int find(String s, String digits, int
index) {

    if (s.length() == digits.length()) {
        System.out.println(s);
        return 1;
    }

    char digit = digits.charAt(index);
    String letters = LETTERS[digit - '1'];
    int val = 0;
    for (char letter : letters.toCharArray())
    {

        val = val + find(s + letter, digits,
index + 1);

    }
    return val;

}

}

```

95.

```

package ArraysMissed;

import java.util.Arrays;
import java.util.Iterator;

public class CommonPrefix {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String[] arr = { "flower", "fluency",
"flow" };
        System.out.println(find(arr));
    }
}

```

```

    }

    static String find(String[] arr) {
        Arrays.sort(arr);
        System.out.println(Arrays.toString(arr));
        int firstLen = arr[0].length();
        int lastLen = arr[arr.length -
1].length();

        int len = firstLen < lastLen ? firstLen :
lastLen;
        int count = 0;

        for (int i = 0; i < len; i++) {
            if (arr[0].charAt(i) == arr[arr.length
- 1].charAt(i)) {
                count++;
            } else {
                break;
            }
        }

        return arr[0].substring(0, count);
    }
}

```

97.

```
package ArraysMissed;
```

```
public class EncodeTheString {
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String s = "aaaabaaabbbbccccdddefefeddd";
        find(s);
    }
}

```



```

    }

    static void find(String s) {
        int start = 0;
        int count = 1;
        char currentChar = 0;
        String op = "";

        for (int i = 0; i < s.length(); i++) {

            currentChar = s.charAt(i);
            if (i + 1 != s.length() && currentChar
== s.charAt(i + 1)) {
                count++;
            } else {
                op = op + currentChar + count;
                count = 1;
                start = i;
            }

        }

        System.out.println(op);

    }

}

```

98.

```
package ArraysMissed;
```

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
```

```
public class PrintListOfAnagram {
```

```
    public static void main(String[] args) {
```

```

        // TODO Auto-generated method stub
        ArrayList<String> list = new
ArrayList<>();
        list.add("ate");
        list.add("eat");
        list.add("tea");
        list.add("bus");
        list.add("sub");
        list.add("cat");

        HashMap<String, List<String>> map = new
HashMap<>();
        for (int i = 0; i < list.size(); i++) {
            String word = list.get(i);
            char[] wordArray = word.toCharArray();
            Arrays.sort(wordArray);
            String sortedWord = new
String(wordArray);
            ArrayList<String> value =
(ArrayList<String>) map.get(sortedWord);

            if (map.containsKey(sortedWord)) {
                value.add(word);
            } else {
                ArrayList<String> str = new
ArrayList<>();
                str.add(word);
                map.put(sortedWord, str);
            }

        }
        System.out.println(map);

    }

}

```

```

package ArraysMissed;

import java.util.Stack;

public class RemoveAdjacentDuplicates {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String ip = "abbbbad";
        System.out.println(find(ip));
    }

    static String find(String ip) {
        Stack<Character> stack = new Stack<>();

        for (char c : ip.toCharArray()) {
            if (!stack.isEmpty() && stack.peek()
== c) {
                stack.pop();
            } else {
                stack.push(c);
            }
        }

        StringBuilder sb = new StringBuilder();
        for (char c : stack) {
            sb.append(c);
        }

        return sb.toString();
    }
}

100.
package ArraysMissed;

```

```
import java.util.Arrays;
```

```
public class
```

```
RemoveDuplicatesFromArrayWithoutDelete {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        int[] arr = { 7, 1, 1, 2, 3, 3, 2, 5, 1,  
4, 5, 7, 7 };  
        // sortAndRemove(arr);  
        withoutSort(arr);  
    }
```

```
    static void sortAndRemove(int[] arr) {  
        Arrays.sort(arr);  
        System.out.println(Arrays.toString(arr));  
        int start = 0;  
        int end = arr.length;  
  
        while (start < end) {  
            if ((start + 1 != end) && arr[start] != arr[start + 1])  
                System.out.print(arr[start] + " ");  
            else if (start + 1 == end) {  
                System.out.print(arr[start] + " ");  
            }  
  
            start++;  
        }  
    }
```

```
    static void withoutSort(int[] arr) {  
        int[] freq = new int[10];  
  
        for (int i = 0; i < arr.length; i++) {  
            freq[arr[i]] = 1;  
        }  
    }
```

```

    }
    for (int i = 0; i < arr.length; i++) {
        if (freq[arr[i]] == 1) {
            System.out.print(arr[i] + " ");
            freq[arr[i]]--;
        }
    }
}
}
}
}

```

101.

```
package ArraysMissed;
```

```
public class reverseWords {
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        reverseWord("Hello how are you");
    }

```

```

    static void reverseWord(String sentence) {
        char[] arr = sentence.toCharArray();
        int start = 0;
        int end = arr.length - 1;

        reverse(arr, start, end);
        start = -1;
        end = -1;

        for (int i = 0; i < arr.length; i++) {

            if (arr[i] != ' ' && start == -1) {
                start = i;
            }

```

```

        if (arr[i] == ' ') {
            end = i - 1;
        } else if (i == arr.length - 1) {
            end = i;
        }
        if (start != -1 && end != -1) {
            reverse(arr, start, end);
            start = -1;
            end = -1;
        }
    }
}

```

```

    System.out.println(new String(arr));

```

```

}

```

```

static void reverse(char[] word, int start,
int end) {
    while (start <= end) {
        char temp = word[start];
        word[start] = word[end];
        word[end] = temp;
        start++;
        end--;
    }
}

```

```

}

```

102.

```

package ArraysMissed;

```

```

import java.util.Arrays;

```

```

public class SmallestWindowSubstring {

```

```

    public static void main(String[] args) {

```

```

// TODO Auto-generated method stub
String s1 = "helloworld";
String s2 = "lor";
System.out.println(find(s1, s2));
}

static int find(String s1, String s2) {
    if (s2.length() > s1.length()) {
        return 0;
    }
    int[] freq = new int[26];
    for (int i = 0; i < s2.length(); i++) {
        freq[s2.charAt(i) - 'a']++;
    }
    int[] freq2 = freq;
    int start = -1;
    int end = -1;
    int minLen = Integer.MAX_VALUE;
    int count = 0;

    for (int i = 0; i < s1.length(); i++) {
        if (freq2[s1.charAt(i) - 'a'] > 0) {

            count++;
            if (start == -1)
                start = i;
            freq2[s1.charAt(i) - 'a']--;
        }
        if (count == s2.length()) {
            end = i;
            if (end - start < minLen) {
                minLen = end - start;
            }
            freq2 = freq;
            start = -1;
        }
    }
}

```

```
    }  
  
    return minLen;  
}  
  
}
```

103.

```
package ArraysMissed;
```

```
public class TwoArraysRotatedToEach {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub
```

```
        String s1 = "hello";
```

```
        String s2 = "lloeh";
```

```
        System.out.println(find(s1, s2));
```

```
    }
```

```
    static boolean find(String s1, String s2) {
```

```
        s1 = s1 + s1;
```

```
        int val = s1.indexOf(s2);
```

```
        if (val == -1) {
```

```
            return false;
```

```
        }
```

```
        return true;
```

```
    }
```

```
}
```

104.

```
package CoinSun;
```

```
import java.util.ArrayList;
```



```

import java.util.HashMap;
import java.util.List;

public class PossibleWaysCoinSumWithMem {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 2, 5 };
        int taraget = 11;
        find(arr, taraget, new
ArrayList<Integer>(), new HashMap<>());
    }

    static ArrayList<Integer> find(int[] arr, int
target, ArrayList<Integer> list,
        HashMap<Integer, ArrayList<Integer>>
map) {

        if (target == 0) {
            System.out.println(list);
            return list;
        }
        if (map.get(target) != null) {
            return map.get(target);
        }
        if (target < 0) {
            return null;
        }

        List<Integer> cu = new ArrayList<>();

        for (int i = 0; i < arr.length; i++) {
            list.add(arr[i]);
            cu = find(arr, target - arr[i], list,
map);
            list.remove(list.size() - 1);
        }
    }
}

```

```
map.put(target, (ArrayList<Integer>) cu);  
return map.get(target);
```

```
}
```

```
}
```

105.

```
package Matrices;
```

```
import java.util.Arrays;
```

```
public class TransposeMatrix {
```

```
    public static void main(String[] args) {
```

```
        // TODO Auto-generated method stub
```

```
        int[][] arr = { { 1, 2, 3 },  
                        { 4, 5, 6 },  
                        { 7, 8, 9 } };
```

```
        transpose(arr);
```

```
        for (int i = 0; i < arr.length; i++) {
```

```
            System.out.println(Arrays.toString(arr[i]));  
        }
```

```
    }
```

```
    static void transpose(int[][] arr) {
```

```
        int rows = arr.length;
```

```
        int col = arr[0].length;
```

```
        for (int i = 0; i < rows; i++) {
```

```
            for (int j = 0; j < col/2; j++) {
```

```
                System.out.println(i+" "+j);
```

```
                int temp = arr[i][j];
```

```
                arr[i][j] = arr[j][i];
```

```
                arr[j][i] = temp;
```

```

        }
    }
}

```

106.

```
package slidingWindow;
```

```
public class FindMaxMulOfSubArrayOfGivenWindow {
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, -2, -2, -4, 5, 6, 1, 1, 8
};

```

```

        int window = 3;
        int max = findMaxMul(arr, window);
        System.out.println(max);

```

```
    }
```

```
static int findMaxMul(int[] arr, int window) {
```

```

    int max = 0;
    int currentMul = 1;

```

```

    for (int i = 0; i < window; i++) {
        currentMul = currentMul * arr[i];
    }

```

```

    max = currentMul;
    for (int i = window; i < arr.length; i++)

```

```

{
    currentMul = currentMul * arr[i] /
arr[i - window];

```

```

        if (currentMul > max) {
            max = currentMul;

```

```

        }

    }

    return max;
}

```

107.

```
package slidingWindow;
```

```
public class
FindMaxOfMulOfSubArrayWhenitHasZerosWithGivenWindo
w {
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 3, 0, -3, 5, 3, 6, 7 };
        int window = 3;
        int max = find(arr, window);
        System.out.println(max);
    }

```

```

    static int find(int[] arr, int window) {

        int max = 0;
        int currentMul = 1;
        for (int i = 0; i < window; i++) {
            currentMul = currentMul * arr[i];
        }
        max = currentMul;

        for (int i = window; i < arr.length; i++)
        {
            if (arr[i] == 0)
                continue;
            if (arr[i - window] == 0) {

```

```

        currentMul = currentMul * arr[i] /
1;
    } else
        currentMul = currentMul * arr[i] /
arr[i - window];

    if (currentMul > max) {
        max = currentMul;
    }
}

return max;
}
}

```

108.

```
package slidingWindow;
```

```

public class FindMaxOfSumSubArrayElements {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 2, -3, 0 };
        int max = find(arr);
        System.out.println(max);

    }

    static int find(int[] arr) {

        int currentSum = 0;
        int max = Integer.MIN_VALUE;

        for (int i = 0; i < arr.length; i++) {
            currentSum = Math.max(arr[i],
currentSum + arr[i]);

```

```

        max = Math.max(max, currentSum);
    }

    return max;
}
}

```

109.

```

package slidingWindow;

public class FindMaxSumOfSubArrayOfGivenWindow {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 1, 3, -1, -3, 5, 3, 6, -7 };
        int window = 3;
        int max = find(arr, window);
        System.out.println(max);
    }

    static int find(int[] arr, int window) {

        int max = 0;
        int currentSum = 0;
        for (int i = 0; i < window; i++) {
            currentSum = currentSum + arr[i];
        }
        max = currentSum;

        for (int i = window; i < arr.length; i++)
        {
            currentSum = currentSum + arr[i] -
arr[i - window];

```

```

        if (currentSum > max)
            max = currentSum;
    }

    return max;
}

}

110.
package slidingWindow;

public class MinOfAddOfSubArrayWithGivenWIndow {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        int[] arr = { 1, 3, 2, -1, 7, -8, 9, 3, -2
};
        int windowSize = 3;
        int min = find(arr, windowSize);
        System.out.println(min);

    }

    static int find(int[] arr, int windowSize) {
        int currentSum = 0;
        int min = 0;

        for (int i = 0; i < windowSize; i++) {
            currentSum = currentSum + arr[i];
        }

        min = currentSum;

        for (int i = windowSize; i < arr.length;
i++) {

```

```

        currentSum = currentSum + arr[i] -
arr[i - windowSize];
        min = Math.min(currentSum, min);
    }

    return min;
}
}

```

111.

```
package strings;
```

```
public class AnagramCheck {
```

```
    public static void main(String[] args) {
        // TODO Auto-generated method stub

```

```
        String s1 = "a";
        String s2 = "a";

```

```
        boolean isAnagram = find(s1, s2);

```

```
        System.out.println(isAnagram);

```

```
    }

```

```
    static boolean find(String s1, String s2) {

```

```
        if (s1.length() != s2.length()) {
            return false;
        }

```

```
        int[] freq = new int[26];

```

```
        for (int i = 0; i < s1.length(); i++) {

```

```
            int index = s1.charAt(i) - 'a';

```



```

        freq[index]++;
    }

    for (int i = 0; i < s2.length(); i++) {
        int index = s2.charAt(i) - 'a';
        freq[index]--;
    }

    for (int i = 0; i < freq.length; i++) {
        if (freq[i] != 0) {
            return false;
        }
    }

    return true;
}
}

```

112.

```
package strings;
```

```

public class FindFreqOfString {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        String s =
"aaabbbbbccccddabcdhjgdfghjaddadajjdajdahjdjhjah
jadhjdahjdahjjhdajdhahjdajdhajhdajdhadjhadhjdjhdjh
dajhdajhdajhdahjdajhdajh";
        find(s);
    }
}

```

```

static void find(String input) {

    int[] frq = new int[26];

    for (int i = 0; i < input.length(); i++) {
        char c = input.charAt(i);
        int index = c - 'a';
        frq[index]++; // frq[index] =
frq[index] +1
    }

    for (int i = 0; i < frq.length; i++) {
        if (frq[i] > 0) {
            System.out.println((char) (i + 'a')
+ " " + frq[i]);
        }

    }

}
}

```

113.

```
package strings;
```

```
public class FirstDuplicate {
```

```
    public static void main(String[] args) {
        // TODO Auto-generated method stub
```

```
        String s = "abcddc";
        System.out.println(find(s));
```

```
    }
```

```
    static char find(String s) {
```

```

        int[] freq = new int[26];

        for (int i = 0; i < s.length(); i++) {
            int index = s.charAt(i) - 'a';
            freq[index]++;

            if (freq[index] > 1) {
                return (char) (index + 'a');
            }
        }

        return 0;
    }
}

```

114.

```
package strings;
```

```
public class PalindromeCheck {
```

```
    public static void main(String[] args) {
        // TODO Auto-generated method stub

```

```
        String ip = "1212";

```

```
        boolean isPalindrome = find(ip);
        System.out.println(isPalindrome);
    }

```

```
    static boolean find(String ip) {

```

```
        int start = 0;
        int end = ip.length() - 1;

```

```
        while (start <= end) {

```

```

        if (ip.charAt(start) !=
ip.charAt(end)) {
            return false;
        }
        start++;
        end--;

    }

    return true;
}
}

```

115.

```
package strings;
```

```
public class
```

```
RemoveMinimumNoOfCharctersToMakeAllEqual {
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        String s = "abacad";
        System.out.println(calculateRemove(s));
    }

```

```
static int calculateRemove(String s) {
```

```
    int[] freq = new int[26];
```

```

    for (int i = 0; i < s.length(); i++) {
        int index = s.charAt(i) - 'a';
        freq[index]++;
    }

```

```
int max = Integer.MIN_VALUE;
```

```

        for (int i = 0; i < freq.length; i++) {
            if (freq[i] > max) {
                max = freq[i];
            }
        }

        return s.length() - max;
    }
}

```

116.

```

package strings;

public class RemoveTrailingZeros {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        String s = "1230400";
        find(s);
    }

    static void find(String s) {

        int firstNonZeroIndex = 0;

        for (int i = s.length() - 1; i >= 0; i--)
        {
            if (s.charAt(i) > '0') {
                firstNonZeroIndex = i;
                break;
            }
        }
    }
}

```

```
        System.out.println(s.substring(0,
firstNonZeroIndex + 1));
```

```
    }
```

```
}
```

117.

```
package strings;
```

```
public class StringReverse {
```

```
    public static void main(String[] args) {
        // TODO Auto-generated method stub
```

```
        String ip = "abc";
```

```
        String op = reverse(ip);
        System.out.println(op);
```

```
    }
```

```
    static String reverse(String ip) {
        char[] arr = ip.toCharArray();
        int start = 0;
        int end = arr.length - 1;
```

```
        while (start <= end) {
            char temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
```

```
        }
```

```
        ip = new String(arr);
```

```

        return ip;
    }
}

118.
package arrays;

import java.util.Arrays;

public class MergeSort {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] arr = { 10, 15, 20, 25, 35, 30,
11 };
        int[] res = new int[arr.length];
        mergesort(arr, res, 0, arr.length - 1);
        System.out.println(Arrays.toString(res));

    }

    static void mergesort(int[] arr, int[] res,
int low, int high) {
        if (low == high) {
            return;
        }
        int mid = (low + high) / 2;
        mergesort(arr, res, low, mid);
        mergesort(arr, res, mid + 1, high);
        merge(arr, res, low, mid, high);

    }

    static void merge(int[] arr, int[] res, int
low, int mid, int high) {
        int k = low, i = low, j = mid + 1;

```

```

    while (i <= mid & j <= high) {
        if (arr[i] <= arr[j]) {
            res[k++] = arr[i++];
        } else {
            res[k++] = arr[j++];
        }
    }
    while (i <= mid) {
        res[k++] = arr[i++];
    }
    while (j <= high) {
        res[k++] = arr[j++];
    }
    //System.out.println(Arrays.toString(res));
    for (i = low; i <= high; i++) {
        arr[i] = res[i];
    }
}
}

```

119.